

Feuille de TD n° 1

Bases de la programmation – Les conditions

Rappels Python

Afficher. L'instruction `print` affiche ses arguments, séparés par des espaces :

```
A = 12
print("Bonjour")
print(A)
print("La valeur contenue dans la variable A est", A)
```

Ce qui s'affiche à l'écran :

```
Bonjour
12
La valeur contenue dans la variable A est 12
```

Lire une valeur au clavier. L'instruction `input` affiche un message, attend que l'utilisateur tape une valeur, puis renvoie cette valeur sous la forme d'une *chaîne de caractères*. Pour obtenir un nombre, il faut la convertir avec `int` (entier) ou `float` (réel) :

```
a = input("Entrez une valeur : ")      # a est une chaîne de caractères
n = int(input("Entrez un entier : "))   # n est un entier
x = float(input("Entrez un réel : "))   # x est un réel
```

Racine carrée. La fonction `sqrt` du module `math` calcule une racine carrée :

```
from math import sqrt
print(sqrt(16))                        # affiche 4.0
```

Exercice 1 Comparaison de deux entiers

Écrire un programme qui demande à l'utilisateur deux entiers, stockés dans deux variables `a` et `b`, et qui affiche « plus grand » si `a` est strictement supérieur à `b`. Dans le cas contraire, le programme affiche « plus petit ou égal ».

Modifier ensuite le programme afin qu'il affiche l'un des trois messages suivants : « plus grand », « plus petit » ou « égal ».

Exercice 2 Aire d'un rectangle

Écrire un programme qui demande à l'utilisateur une longueur et une largeur (deux nombres réels), stockées dans deux variables `longueur` et `largeur`. La longueur doit être supérieure ou égale à la largeur.

Si cette condition est respectée, le programme affiche l'aire du rectangle correspondant. Sinon, il affiche le message « Données incohérentes ».

Exercice 3 Contrôle de saisie

Écrire un programme qui lit six entiers et les stocke dans six variables : **a**, **b**, **c**, **d**, **e** et **f**.

Si au moins trois de ces variables sont non nulles, le programme affiche « La saisie est correcte ». Sinon, il affiche « La saisie est incorrecte ».

Exercice 4 Tri de trois nombres

Écrire un programme qui lit trois nombres et les stocke dans trois variables **a**, **b** et **c**. Le programme doit ensuite réordonner leur contenu de manière à obtenir :

$$a \leq b \leq c.$$

Afficher les trois variables à la fin du programme afin de vérifier le résultat.

Exercice 5 Minimum, maximum et moyenne

Écrire un programme qui permet de saisir dix valeurs réelles et qui affiche la plus petite valeur, la plus grande valeur et la moyenne des dix valeurs. Les boucles n'ayant pas encore été vues, on répétera dix fois le bloc de saisie. Le programme ne doit utiliser que **quatre variables** : la valeur saisie, le minimum, le maximum et la somme.

Exercice 6 Distance entre deux points

Écrire un programme qui saisit les coordonnées entières de deux points du plan et qui affiche la distance les séparant sous la forme d'un nombre réel (voir les rappels pour la racine carrée).

Exercice 7 Triangle rectangle

Écrire un programme qui saisit les coordonnées entières de trois points du plan et qui indique si ces trois points forment un triangle rectangle. On utilisera le théorème de Pythagore sur les carrés des longueurs des côtés (aucune racine carrée n'est nécessaire). Le programme affiche « Triangle rectangle » ou « Pas un triangle rectangle » ; si deux des points sont identiques, il affiche « Deux points sont confondus ».

Exercice 8 Calculatrice

Écrire un programme qui demande à l'utilisateur une opération parmi **+**, **-**, ***** et **/**, puis deux opérandes entiers. Le programme affiche le résultat de l'opération appliquée aux deux opérandes ; pour la division, il affiche le quotient entier (**//**). Si l'opération n'est pas l'une des quatre, le programme affiche « Opération inconnue ».

Attention : pour la division, pensez à interdire la division par zéro.

Exemple d'exécution :

```
Entrez l'opération : *
Entrez le premier opérande : 3
Entrez le deuxième opérande : 4
Le résultat de 3 * 4 est 12
```

Exercice 9 Équation du second degré

Écrire un programme qui demande trois valeurs réelles **a**, **b** et **c**, puis affiche les solutions réelles de l'équation

$$ax^2 + bx + c = 0.$$

Si **a** est nul, le programme affiche « a doit être différent de zéro » et s'arrête. Sinon, selon le signe du discriminant $\Delta = b^2 - 4ac$, il affiche « Pas de solution réelle » ($\Delta < 0$), la solution double ($\Delta = 0$) ou les deux solutions ($\Delta > 0$).

Exercice 10 Chiffres d'un nombre

Écrire un programme qui reçoit un entier positif à quatre chiffres et qui affiche ses quatre chiffres, séparés par des espaces.

Le programme doit d'abord vérifier que le nombre contient bien quatre chiffres. Dans le cas contraire, il affiche : « Le nombre ne contient pas 4 chiffres ».

Exemple : pour 1234, le programme affiche 1 2 3 4.

Feuille de TD n° 1 – Corrigé

Bases de la programmation – Les conditions

Remarque générale : les solutions proposées n'utilisent que les notions de la feuille : lecture au clavier avec `input`, conversions `int` / `float`, affichage avec `print`, instructions conditionnelles `if` / `elif` / `else` et `sqrt`. Pour l'exercice 5, une version avec boucle est donnée en complément, à revoir après le TD n° 2. D'autres solutions correctes sont bien sûr possibles.

Exercice 1 Comparaison de deux entiers

Version 1 : deux messages

```
a = int(input("Entrez un premier entier : "))
b = int(input("Entrez un deuxième entier : "))

if a > b:
    print("plus grand")
else:
    print("plus petit ou égal")
```

Version 2 : trois messages

```
a = int(input("Entrez un premier entier : "))
b = int(input("Entrez un deuxième entier : "))

if a > b:
    print("plus grand")
elif a < b:
    print("plus petit")
else:
    print("égal")
```

Remarque : `elif` se lit « sinon, si ». Les trois cas `a > b`, `a < b` et `a == b` sont testés dans l'ordre ; le dernier `else` correspond forcément à l'égalité.

Exercice 2 Aire d'un rectangle

```
longueur = float(input("Entrez la longueur : "))
largeur = float(input("Entrez la largeur : "))

if longueur >= largeur:
    print("L'aire du rectangle est", longueur * largeur)
else:
    print("Données incohérentes")
```

Remarque : `float` accepte aussi bien 3 que 2.5. Le test `longueur >= largeur` se fait *avant* tout calcul.

Exercice 3 Contrôle de saisie

```
a = int(input("Entrez a : "))
b = int(input("Entrez b : "))
c = int(input("Entrez c : "))
d = int(input("Entrez d : "))
e = int(input("Entrez e : "))
f = int(input("Entrez f : "))

nb = 0          # nombre de variables non nulles
if a != 0:
    nb = nb + 1
if b != 0:
    nb = nb + 1
if c != 0:
    nb = nb + 1
if d != 0:
    nb = nb + 1
if e != 0:
    nb = nb + 1
if f != 0:
    nb = nb + 1

if nb >= 3:
    print("La saisie est correcte")
else:
    print("La saisie est incorrecte")
```

Remarque : En Python, True vaut 1 et False vaut 0 : le comptage peut s'écrire en une ligne, $nb = (a \neq 0) + (b \neq 0) + (c \neq 0) + (d \neq 0) + (e \neq 0) + (f \neq 0)$.

Exercice 4 Tri de trois nombres

```
a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))

# On amène le plus petit des trois dans a
if a > b:
    tmp = a
    a = b
    b = tmp
if a > c:
    tmp = a
    a = c
    c = tmp
# Il reste à ordonner b et c
if b > c:
    tmp = b
    b = c
    c = tmp

print(a, b, c)
```

Remarque : Après les deux premiers tests, a contient le plus petit des trois nombres ; le troisième test ordonne b et c. L'échange de deux variables passe par une variable temporaire tmp ; Python permet aussi d'écrire directement a, b = b, a.

Exercice 5 Minimum, maximum et moyenne

Sans boucle (bloc de saisie répété)

```
# La première valeur initialise le minimum, le maximum et la somme
x = float(input("Entrez une valeur : "))
mini = x
maxi = x
somme = x

# Deuxième valeur
x = float(input("Entrez une valeur : "))
if x < mini:
    mini = x
if x > maxi:
    maxi = x
somme = somme + x

# Troisième valeur
x = float(input("Entrez une valeur : "))
if x < mini:
    mini = x
if x > maxi:
    maxi = x
somme = somme + x

# ... même bloc, recopié à l'identique, pour les valeurs 4 à 10 ...

print("Plus petite valeur :", mini)
print("Plus grande valeur :", maxi)
print("Moyenne :", somme / 10)
```

Avec une boucle (à réécrire après le TD n° 2)

```
x = float(input("Entrez une valeur : "))
mini = x
maxi = x
somme = x
for i in range(9):
    # les neuf valeurs suivantes
    x = float(input("Entrez une valeur : "))
    if x < mini:
        mini = x
    if x > maxi:
        maxi = x
    somme = somme + x

print("Plus petite valeur :", mini)
print("Plus grande valeur :", maxi)
print("Moyenne :", somme / 10)
```

Remarque : Les quatre variables sont `x`, `mini`, `maxi` et `somme`. Initialiser `mini` et `maxi` avec la première valeur saisie évite d'avoir à choisir une valeur de départ arbitraire. Le bloc de traitement est strictement le même pour les neuf valeurs suivantes : c'est exactement ce qu'une boucle permet d'écrire une seule fois.

Exercice 6 Distance entre deux points

```
from math import sqrt

x1 = int(input("Abscisse du premier point : "))
y1 = int(input("Ordonnée du premier point : "))
x2 = int(input("Abscisse du deuxième point : "))
y2 = int(input("Ordonnée du deuxième point : "))

d = sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2)
print("La distance entre les deux points est", d)
```

Remarque : $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$. La fonction `sqrt` provient du module `math` ; on peut aussi écrire `import math` puis `math.sqrt(...)`.

Exercice 7 Triangle rectangle

```
xa = int(input("Abscisse de A : "))
ya = int(input("Ordonnée de A : "))
xb = int(input("Abscisse de B : "))
yb = int(input("Ordonnée de B : "))
xc = int(input("Abscisse de C : "))
yc = int(input("Ordonnée de C : "))

# Carrés des longueurs des trois côtés (des entiers : pas de racine carrée)
ab = (xb - xa) ** 2 + (yb - ya) ** 2
bc = (xc - xb) ** 2 + (yc - yb) ** 2
ca = (xa - xc) ** 2 + (ya - yc) ** 2

if ab == 0 or bc == 0 or ca == 0:
    print("Deux points sont confondus")
elif ab == bc + ca or bc == ab + ca or ca == ab + bc:
    print("Triangle rectangle")
else:
    print("Pas un triangle rectangle")
```

Remarque : Le triangle est rectangle si le carré du plus grand côté est égal à la somme des carrés des deux autres (théorème de Pythagore). Comme les coordonnées sont entières, les carrés des longueurs sont des entiers : on les compare exactement, sans racine carrée ni arrondi. Le premier test écarte le cas de deux points confondus.

Exercice 8 Calculatrice

```
op = input("Entrez l'opération : ")
a = int(input("Entrez le premier opérande : "))
b = int(input("Entrez le deuxième opérande : "))

if op == "+":
    print("Le résultat de", a, "+", b, "est", a + b)
elif op == "-":
    print("Le résultat de", a, "-", b, "est", a - b)
elif op == "*":
    print("Le résultat de", a, "*", b, "est", a * b)
elif op == "/":
    if b == 0:
        print("Division par zéro impossible")
    else:
        print("Le résultat de", a, "/", b, "est", a // b)
else:
    print("Opération inconnue")
```

Remarque : Le résultat devant être entier, la division utilise le quotient entier `//` (`7 // 2` vaut 3); l'opérateur `/` donnerait un réel (`3.5`). Le test `b == 0` est placé avant la division. Un opérateur inconnu est également traité.

Exercice 9 Équation du second degré

```
from math import sqrt

a = float(input("Entrez a : "))
b = float(input("Entrez b : "))
c = float(input("Entrez c : "))

if a == 0:
    print("a doit être différent de zéro")
else:
    delta = b ** 2 - 4 * a * c
    if delta < 0:
        print("Pas de solution réelle")
    elif delta == 0:
        print("Une solution double :", -b / (2 * a))
    else:
        x1 = (-b - sqrt(delta)) / (2 * a)
        x2 = (-b + sqrt(delta)) / (2 * a)
        print("Deux solutions :", x1, "et", x2)
```

Remarque : Discriminant $\Delta = b^2 - 4ac$: pas de solution réelle si $\Delta < 0$, une solution double $x = -\frac{b}{2a}$ si $\Delta = 0$, deux solutions $x = \frac{-b \pm \sqrt{\Delta}}{2a}$ si $\Delta > 0$.

Exercice 10 Chiffres d'un nombre

```
n = int(input("Entrez un nombre à 4 chiffres : "))

if n < 1000 or n > 9999:
    print("Le nombre ne contient pas 4 chiffres")
else:
    milliers = n // 1000
    centaines = (n // 100) % 10
    dizaines = (n // 10) % 10
    unites = n % 10
    print(milliers, centaines, dizaines, unites)
```

Remarque : Un entier positif a quatre chiffres s'il est compris entre 1000 et 9999. Les chiffres s'obtiennent par divisions entières (//) et restes (%) successifs. Autre solution : convertir le nombre en chaîne, `s = str(n)`, puis afficher `s[0]`, `s[1]`, `s[2]`, `s[3]`.

Feuille de TD n° 2

Bases de la programmation – Les boucles

Rappels Python

Répéter tant qu'une condition est vraie. La boucle `while` exécute son bloc tant que la condition est vraie ; elle est typiquement utilisée pour une saisie contrôlée :

```
n = int(input("Entrez un entier positif : "))
while n < 0:                                # tant que n est négatif...
    n = int(input("Entrez un entier positif : "))
```

Répéter un nombre de fois connu. La boucle `for` parcourt les valeurs de `range` : `range(5)` donne 0, 1, 2, 3, 4 et `range(2, 6)` donne 2, 3, 4, 5 (la borne de fin est exclue) :

```
for i in range(1, 4):
    print("Tour numéro", i)                # affiche Tour numéro 1, puis 2, puis 3
```

Afficher sans passer à la ligne. `print("*", end="")` affiche une étoile sans retour à la ligne ; `print()` seul passe à la ligne. Une chaîne peut aussi être répétée : `"*" * 5` vaut `"*****"`.

Exercice 1 Saisie contrôlée

Écrire un programme qui demande à l'utilisateur de lui fournir un nombre entier compris entre 0 et 99 (inclus), et ceci jusqu'à ce que la réponse soit satisfaisante. Le dialogue avec l'utilisateur se présentera ainsi :

```
Donnez un entier entre 0 et 99 : 453
Donnez un entier entre 0 et 99 : 25
Merci pour le nombre 25
```

Exercice 2 Table de multiplication

Écrire un programme qui demande à l'utilisateur un nombre `n` compris entre 2 et 9 (inclus) et qui affiche sa table de multiplication, de $1 \times n$ à $10 \times n$. La saisie du nombre est bloquante : tant que le nombre n'est pas compris entre 2 et 9, le programme redemande une valeur.

Exemple d'exécution :

```
Entrez un nombre entre 2 et 9 : 12
Entrez un nombre entre 2 et 9 : 7
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Exercice 3 Moyenne d'une suite de valeurs

- A. Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies. La saisie s'arrête quand l'utilisateur donne la valeur 0.

- B.** Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs saisies, la valeur minimale et la valeur maximale. Les trois valeurs doivent être calculées en utilisant une seule boucle. La saisie s'arrête quand l'utilisateur donne la valeur 0.

Dans les deux cas, la valeur 0 qui termine la saisie n'est pas prise en compte. Si l'utilisateur tape 0 dès le départ, le programme affiche « Aucune valeur saisie ».

Exercice 4 Moyennes des valeurs paires et impaires

Écrire un programme qui saisit une suite de valeurs entières non nulles. À la fin de la saisie, le programme doit afficher la moyenne des valeurs paires et la moyenne des valeurs impaires. La saisie s'arrête quand l'utilisateur donne la valeur 0.

Si aucune valeur paire (respectivement impaire) n'a été saisie, le programme affiche « Aucune valeur paire » (respectivement « Aucune valeur impaire ») à la place de la moyenne correspondante.

Attention : la division par zéro est une opération interdite.

Exercice 5 Le juste prix

- A.** Écrire un programme qui permet de déterminer le juste prix (juste prix = 42). L'utilisateur saisit un nombre entier. Le programme répond « inférieur » (le juste prix est inférieur au prix proposé) si le prix indiqué est supérieur à 42, il répond « supérieur » (le juste prix est supérieur au prix proposé) si le prix indiqué est inférieur à 42, et il répond « BRAVO ! » s'il trouve le prix exact. Tant que le prix n'est pas le bon, l'utilisateur doit retenter sa chance en saisissant un nouveau nombre.

Exemple d'exécution :

```
Votre prix : 50
inférieur
Votre prix : 30
supérieur
Votre prix : 42
BRAVO !
```

- B.** Modifier le programme précédent afin que le juste prix soit tiré au hasard entre 1 et 100 (inclus), et non codé en dur à 42 ; il sera évidemment inconnu de l'utilisateur.

De plus, le programme doit, à la fin de chaque partie, afficher le nombre de coups de la partie (un coup est une proposition de l'utilisateur) et demander à l'utilisateur s'il veut continuer le jeu. Si la réponse est exactement « o », le programme lance une nouvelle partie ; sinon, il affiche le nombre de coups moyen sur l'ensemble des parties jouées, puis « À bientôt », et s'arrête.

Indication : après `from random import randint`, l'appel `randint(1, 100)` renvoie un entier tiré au hasard entre 1 et 100 (inclus).

Exercice 6 Rectangle d'étoiles

Le programme a besoin de deux entrées, que l'utilisateur doit fournir : la largeur (nombre de colonnes) et la hauteur (nombre de lignes) du rectangle.

- A. Écrire un programme qui dessine un rectangle *plein* d'étoiles `*`.

Exemple d'exécution :

```
Largeur : 5
Hauteur : 3
*****
*****
*****
```

- B. Modifier le programme pour qu'il dessine un rectangle *vide* : seules les étoiles du contour sont affichées, l'intérieur est rempli d'espaces.

Exemple d'exécution :

```
Largeur : 5
Hauteur : 4
*****
*      *
*      *
*      *
*****
```

Exercice 7 Dessiner un X

Écrire un programme qui permet de dessiner un **X** avec des `*`. Le programme a besoin d'une entrée : la taille de la forme, qui occupe `taille` lignes et `taille` colonnes. La taille doit être un nombre entier impair compris entre 15 et 20 (inclus), c'est-à-dire 15, 17 ou 19. Tant que le nombre saisi ne vérifie pas la condition, le programme redemande la valeur de la taille.

Exemple pour une taille de 5 (la taille demandée est plus grande) :

```
*      *
 *     *
  *
 *     *
*      *
```

Exercice 8 Factorielle

Écrire un programme qui calcule et affiche la factorielle d'un entier $n \geq 0$:

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 1;$$

par définition, $0! = 1$. Si la valeur saisie est négative, le programme la redemande.

Exercice 9 Nombres premiers

Un entier $n \geq 2$ est dit premier si ses seuls diviseurs sont 1 et lui-même ; 1 n'est pas premier. Écrire un programme qui saisit un entier et qui affiche « PREMIER » s'il est premier et « PAS PREMIER » sinon.

Modifier le programme pour qu'il saisisse deux entiers positifs **a** et **b**, dans n'importe quel ordre, et qu'il affiche tous les nombres premiers compris entre les deux, bornes incluses.